Random Forest Algorithm

Ensembles · Decision Trees · Bagging · Feature Importance

What We'll Cover Today

01
From Trees to Forests
The core insight — why many trees beat one

02
How Random Forest Works
Bagging, bootstrapping & random subsets

03
Key Hyperparameters
n_estimators, max_depth, max_features

04
Feature Importance
Which features matter most?

05
Random Forest in Python
scikit-learn implementation

06
Pros & Cons
When to use RF — and when NOT to

Quick Recap: Decision Trees

Root Node

The starting point. Ask the most informative question about the data.

→

Branch Nodes

Each internal node asks another yes/no question, splitting the data further.

→

Leaf Nodes

Final decisions. Each leaf gives a class label or predicted value.

The Problem with a Single Tree:
A single deep tree OVERFITS — it memorizes training data noise and performs poorly on new data. It's brittle: small changes in training data create completely different trees!

The Solution — Random Forest:
Build MANY trees, each on a different random sample of data, and combine their predictions. Errors cancel out — accuracy goes up! "The wisdom of crowds."

What is Random Forest? The Core Idea

Real-Life Analogy: Ask 500 experts instead of 1. Even if each expert is sometimes wrong, the majority vote is almost always right.

Random Forest builds hundreds of decision trees on random data subsets, then combines their answers into one final, robust prediction.

1

Bootstrap Sampling

Draw N random samples with replacement from the training data → N slightly different datasets

2

Build Many Trees

Train one decision tree on each bootstrapped dataset — each tree is different!

3

Random Feature Split

At each node, only consider √p random features (not all). Reduces correlation.

4

Aggregate Predictions

Classification → majority vote
Regression → average
Final answer is more accurate!

Bagging — Bootstrap AGGregatING

What is Bootstrapping?

Create a new dataset of N rows by sampling the ORIGINAL dataset with replacement. Some rows appear multiple times, some don't appear at all (~37% left out each time).

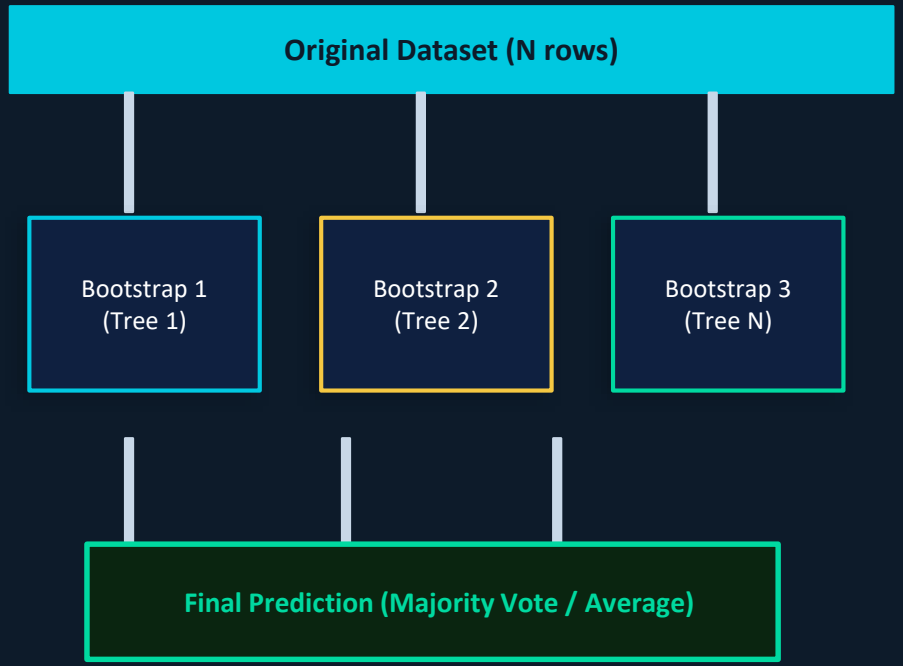
What is Aggregating?

Combine all tree predictions:

- Classification → majority vote (most common class)
- Regression → mean of all predictions

Out-of-Bag (OOB) Score

~37% of samples not used per tree = free validation set! OOB score estimates generalization without a separate test split.

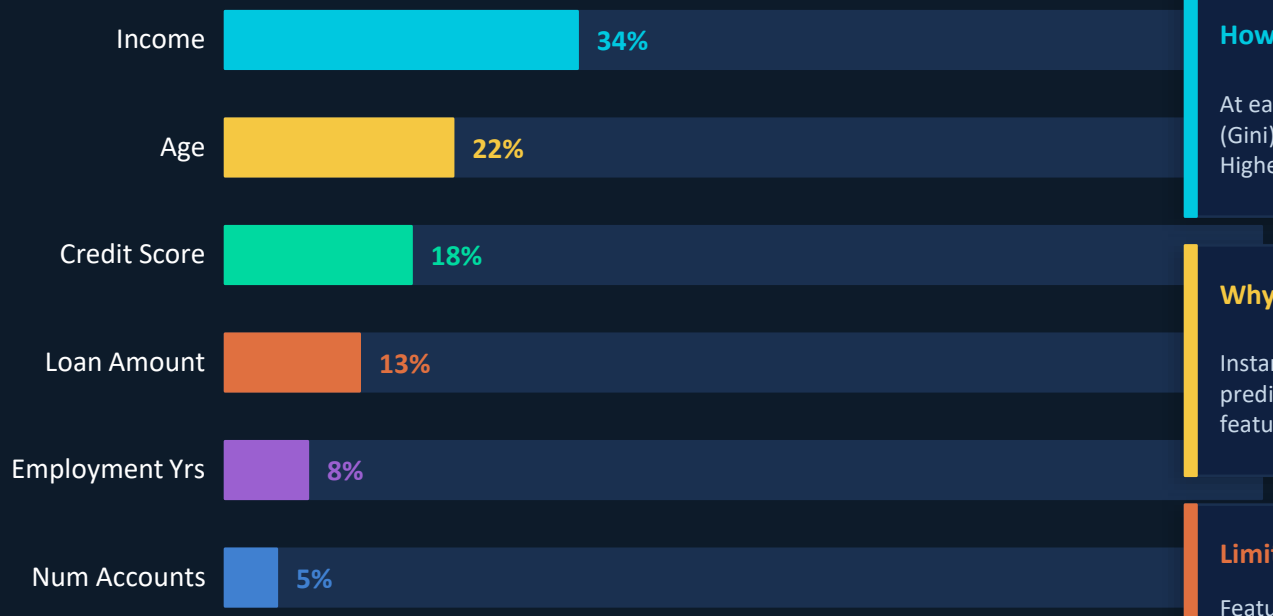


Key Hyperparameters — Tuning Random Forest

n_estimators Number of trees in the forest	Default: 100	💡 More trees = better accuracy & stability, but slower. Usually plateau after 200–500. Start with 100, increase if needed.
max_depth Maximum depth of each tree	Default: None (unlimited)	💡 None lets trees grow until pure leaves (risk of overfit). Set 5–20 to control complexity. Tune with cross-validation.
max_features # features considered at each split	Default: √p (classification)	💡 'sqrt' = √total features — the key randomness! Lower → more diverse trees. Try 'sqrt', 'log2', or a fraction like 0.5.
min_samples_split Min samples needed to split a node	Default: 2	💡 Increase (e.g. 5–20) to prevent trees from splitting on tiny groups — reduces overfitting on noisy data.
oob_score Use out-of-bag samples to evaluate	Default: False	💡 Set True for a FREE generalization estimate without a separate validation set. Very useful in practice!

Feature Importance — Which Features Matter Most?

Random Forest gives every feature an importance score — a huge advantage over KNN!



How it's calculated

At each split, measure how much impurity (Gini) is reduced. Average across all trees. Higher reduction = more important feature.

Why it's powerful

Instantly shows which inputs drive predictions. Use to remove irrelevant features and speed up inference.

Limitation: watch for bias

Features with many unique values (high cardinality) can appear artificially important. Use Permutation Importance for robust results.

Random Forest in Python — scikit-learn

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1. Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 2. Scale features (good practice, less critical for RF)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

# 3. Train the model
rf = RandomForestClassifier(n_estimators=200, max_depth=10,
                            oob_score=True, random_state=42)
rf.fit(X_train, y_train)

# 4. Evaluate
print(f'Test Accuracy : {rf.score(X_test, y_test):.2f}')
print(f'OOB Score     : {rf.oob_score_:.2f}')

# 5. Feature importance
importances = rf.feature_importances_   # array of scores
```

`n_estimators=200`

200 trees — good balance of accuracy & speed.

`max_depth=10`

Limit depth to prevent overfitting. Tune per dataset.

`oob_score=True`

Free validation — no separate split needed. Compare to test accuracy.

`feature_importances_`

Array of scores 0→1 summing to 1.0. Higher = more important.

Random Forest vs KNN — Head-to-Head

	Random Forest	KNN
Training Speed	Fast	Instant (lazy learner)
Prediction Speed	Fast	Slow O(n·d)
Memory Usage	Moderate	High (stores all data)
Accuracy (general)	High ✓	Moderate
Handles Missing Data	Yes (with care)	No — must impute
Feature Scaling Needed	No ✓	Yes — critical!
Feature Importance	Built-in ✓	Not available
Interpretability	Medium	High (intuitive)
Handles High Dimensions	Yes ✓	Struggles (curse of dim.)
Best for	Most classification/regression tasks	Small datasets, baselines

Random Forest — Advantages & Disadvantages

ADVANTAGES

✓ High Accuracy
One of the best off-the-shelf algorithms. Often beats single trees, logistic regression, and KNN without tuning.

✓ Robust to Overfitting
Averaging many trees cancels out individual overfitting — works well even with deep trees.

✓ No Feature Scaling
Distance-based? Not here. Features at any scale work fine — no normalization needed!

✓ Feature Importance
Tells you WHICH features drive predictions — invaluable for business insights and feature selection.

✓ Handles Missing Data
Can handle some missing values; handles mixed data types (numerical + categorical) well.

DISADVANTAGES

✗ Slow to Predict at Scale
Hundreds of trees × all features = slow inference for real-time applications with tight latency requirements.

✗ Memory Hungry
Stores all N trees in memory. Can be large for 500+ trees on high-dimensional data.

✗ Less Interpretable
Can't easily explain a single prediction (use SHAP values for explainability).

✗ Slow on Sparse Data
Not ideal for very high-dimensional sparse data (e.g. raw text). SVM or neural nets may be better.

✗ May Overfit on Noise
With unlimited depth and small datasets, still prone to memorizing noise despite bagging.

Decision Guide — When Should You Use Random Forest?

USE Random Forest when:	AVOID Random Forest when:
✓ Dataset has 1,000 to 1,000,000 rows	✗ Data is hundreds of millions of rows
✓ Mix of numerical & categorical features	✗ Real-time prediction < 10ms required
✓ You need a strong baseline quickly	✗ Highly sparse data (e.g. bag-of-words NLP)
✓ Feature importance insights are needed	✗ Full model explainability is legally required
✓ Noisy data or some missing values	✗ GPU acceleration is a must (use XGBoost/NN)
✓ Non-linear patterns in data	✗ Sequential/temporal patterns in data → LSTM

Rule of Thumb: Random Forest is the best all-purpose 'first serious model' after KNN. It often wins Kaggle competitions and production ML pipelines with minimal tuning. Try it before XGBoost or Neural Networks.

Session Summary — Key Takeaways

01

Ensemble = many weak learners

Combine 100s of decision trees, each on a random data/feature subset. Errors cancel out → higher accuracy than any single tree.

02

Bagging prevents overfitting

Bootstrap sampling + aggregation (majority vote/average) gives Random Forest its robustness. ~37% of each sample is left as a free OOB validation set.

03

No feature scaling needed

Unlike KNN, Random Forest is distance-free. Raw feature values work fine. But still encode categoricals and handle missing data.

04

Tune n_estimators & max_depth

More trees = more stable (plateau after ~500). Limit max_depth to control complexity. Use oob_score for cheap evaluation.

05

Feature importance is a bonus

Free ranking of every input feature. Use it for data understanding, feature selection, and stakeholder explanations.

Great work!

You now understand Random Forest.